



Cross-domain digital twin architecture for predictive maintenance via machine learning and Large Language Models[☆]

John Williams^a, Gerald Jones^a, Tom Berg^a, Luke Birt^b, Ashley Stowe^{a,b,c}, Xueping Li^a^{*,*}

^a University of Tennessee, Knoxville, 2233 Volunteer Blvd., Knoxville, 37996, TN, USA

^b Y-12 Security Complex, 301 Bear Creek Rd, Oak Ridge, 37830, TN, USA

^c Oak Ridge Enhanced Technology and Training Center (ORETTC), 3607 Oak Ridge Turnpike, Oak Ridge, 37830, TN, USA

ARTICLE INFO

Keywords:

Predictive maintenance
Digital twin
Game engine
Machine learning
Large Language Models
Augmented reality

ABSTRACT

This research introduces a comprehensive framework for creating and deploying a digital twin platform for continuous monitoring and predictive maintenance within industrial settings. Through utilizing advanced technologies, including Unreal Engine 5, Unity 3D, the Message Queue Telemetry Transport protocol, Random Forest machine learning algorithms, and Large Language Models (LLMs), we establish a platform that digitally reproduces physical equipment and translates digital controls into real-world actions. This facilitates preventive maintenance approaches and improves operational effectiveness. The digital twin platform gathers sensor data from operational equipment, analyzes it using machine learning, and delivers practical insights to prevent potential malfunctions and enhance equipment performance. Moreover, the incorporation of a web portal enables efficient monitoring and access to historical data, educational materials, and equipment status information. Preliminary findings indicate that digital twins can transform industrial equipment management and maintenance methodologies.

1. Introduction

Industrial environments face persistent challenges in maintaining the reliability and efficiency of operational assets. Unplanned downtime, equipment failures, and inefficient maintenance practices can impose substantial costs and disrupt production. In response, digital twins have emerged as a promising approach to enhance asset monitoring, predictive maintenance, and overall operational performance. A digital twin is a virtual representation that mirrors the “structure, context, and behavior of a natural, engineered, or social system”, typically instrumented with sensors and analytics to reflect real-world behavior and support decision-making (National Academies of Sciences, Engineering, and Medicine, 2023).

Over the past decade, advances associated with Industry 4.0 have accelerated the adoption of digital-twin methods (Starly et al., 2023). These advances enable improved model fidelity, reliability, and sustainability. However, many deployments still rely heavily on proprietary software, bespoke development pipelines, and deep domain expertise, which can limit scalability and reproducibility (González-Herbón et al., 2024).

Moreover, recent advances in game-engine technologies have opened new opportunities for predictive maintenance in industrial settings.

Platforms such as Unreal Engine 5 (UE5) (Epic Games, 2024) provide photorealistic rendering, advanced lighting, fluid dynamics, and physics-based simulation that closely approximate operational conditions (Clausen et al., 2022-12-21). These capabilities make game engines compelling hosts for digital twins that ingest real-time data from local and network sources, enabling intuitive visualization and supporting fault prognosis and maintenance planning. In parallel, game engines support iterative development: models and environments can evolve across cycles from early prototyping to high-fidelity replicas of industrial equipment and facilities. Despite growing interest, the use of game engines for digital-twin-driven maintenance prediction remains underrepresented in the literature; further documentation of architectures, trade-offs, and empirical results is warranted. The result of many existing digital twin implementations emphasize accurate state estimation or isolated visualizations, but do not provide an integrated, resource-efficient pathway from heterogeneous sensor data to actionable, operator-ready maintenance guidance across various industrial contexts. This paper presents a practical digital-twin architecture assembled from readily available, largely open-source components and demonstrates how to integrate and tailor them into an end-to-end platform for monitoring and predictive maintenance.

[☆] This article is part of a Special issue entitled: ‘Resilient Systems-CIE51’ published in Computers & Industrial Engineering.

^{*} Corresponding author.

E-mail address: Xueping.Li@utk.edu (X. Li).

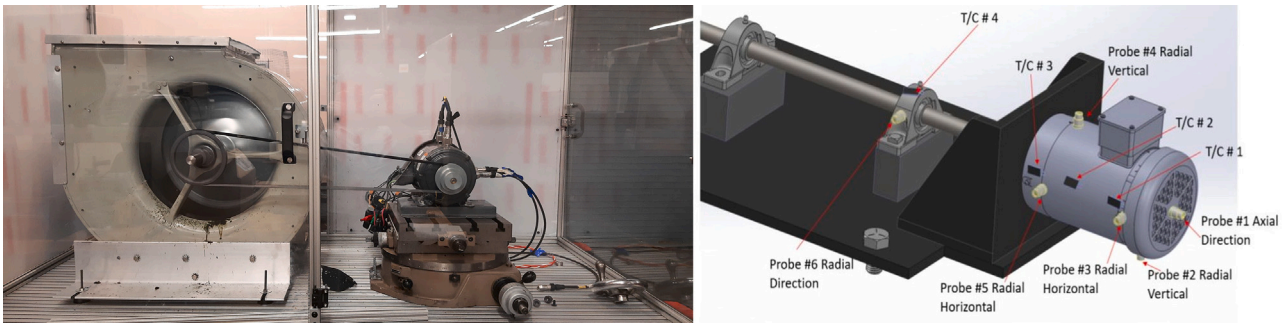


Fig. 1. Belt-drive motor testbed used to collect vibration and MCSA data.

The main contributions of this work are threefold.

- A modular, game-engine-based digital-twin architecture that couples MQTT messaging, on-edge machine learning, and AR/web visualization.
- A supervised learning pipeline (Random Forest) for rotor-fault classification across two testbeds, with discussion of transferability and feature alignment.
- An on-device LLM loop that translates predictions and context into concise, operator-ready instructions for AR and browser clients.

Taken together, these contributions demonstrate an innovative but pragmatic pathway for fusing real-time sensing, classical machine learning, and LLM-based guidance within a game-engine host to close the loop from detection to action. The next section introduces the technology stack and system design that realize this architecture.

The remainder of this paper is organized as follows: Section 2 describes the overall technology stack and system architecture, detailing the messaging, networking, and data-flow mechanisms that facilitate the digital twin. Section 3 presents the implementation and case-study evaluation across two physical testbeds, including predictive-model performance, cross-domain analysis, and software operator interaction. Section 4 discusses the results, limitations, and practical considerations for industrial deployment, and concludes with directions for future research and scalability.

1.1. Enterprise reliability context

This work is motivated by enterprise reliability requirements aimed at improving the effectiveness of maintenance operations in real industrial environments (Li et al., 2023). The study established an experimental testbed (Fig. 1) instrumented to collect vibration measurements and motor current signature analysis (MCSA) data. Supervised machine-learning techniques, including Random Forests, are applied to predict fault states and estimate remaining useful life (RUL). Initial findings indicate that these approaches can accurately classify faults and estimate RUL using standard RUL frameworks.

Additional rotor testing studies that focus on fault prediction encompass various research domains; these include the following:

- Investigations examining failure root causes rather than component-level causation, which are then linked to specific fault classes (Nath et al., 2021). Our approach follows a conventional fault-categorization framework while emphasizing rigorous algorithmic evaluation.
- Work highlighting rotor angular instability as a key factor in fault behavior (Rajapakse et al., 2009). We extend this by explicitly including angular instability as a fault class, thereby expanding the data matrix and revealing cross-class correlations.

- Studies on bearing-fault classification, as demonstrated by Vyas and Satishkumar (2001). While informative, many such efforts stop short of practical industrial deployment and often rely on neural networks; in contrast, our system integrates Random Forests with LLMs for analysis and operator guidance.

Although our work differs from these earlier studies, this research incorporates numerous principles from prior investigations, including fault type categorization, to establish consistency in experimental components, thereby providing a clearer reference framework for interpreting experimental outcomes. This approach also enhances applicability in practice, as this project aims to be implemented beyond academic research contexts. Consequently, choices regarding fault categories and certain data inputs must remain unchanged to preserve utility for subsequent users.

This research employs a rotor testbed to produce substantial data relating operational states to predefined fault classes. Approximately 5000 samples per class are collected and aggregated into a combined dataset to support robust analysis. As outlined by Duan et al. (2023), this experimental platform bridges the physical and data layers of a digital-twin infrastructure—a foundational aspect of twin design.

Building on prior results, we implement a Random Forest classifier to analyze data and forecast faults in the motor system. Operating independently from the UE5 environment, sensor streams are delivered to the classifier over the Message Queue Telemetry Transport (MQTT) protocol. The classifier publishes predicted fault states, which are then forwarded via MQTT to a Large Language Model that produces concise remediation instructions.

1.2. Large language models (LLMs) context

The guidance component leverages Large Language Models (LLMs). LLMs and related deep-learning architectures extend human-AI interaction and can translate heterogeneous inputs (text and numeric context) into operator-ready instructions.

Modern text-generation LLMs benefit from self-supervised learning, removing the need for traditional question-answer supervision (Kang et al., 2020-11). They are trained to predict masked spans, enabling broad world knowledge and instruction-following behavior after fine-tuning.

LLMs have been deployed across several application domains, including the following:

- Text-based assistants for operations and healthcare, where structured forms are pre-populated and work orders or clinical artifacts are drafted (Kamal et al., 2021; Mesko & Topol, 2023).
- Information-retrieval interfaces that augment search with conversational synthesis (Liu et al., 2023).
- Clinical decision support that assists diagnosis and prognosis while keeping a human-in-the-loop (Karabacak & Margetis, 2023).

Given their broad applicability, LLMs are well-suited for turning model predictions and sensor context into succinct, actionable guidance in our setting. Contemporary digital twins not only model and track asset state but also enable intelligent assessment and control with feedback to the physical system (Sun et al., 2024). Our LLM component operationalizes this loop by translating predictions into clear procedural steps for AR and web clients.

With the problem framing and AI roles established, the next section details the technology stack and system design that instrument the physical testbeds, distribute data and predictions, and surface guidance in AR and web interfaces.

2. Technology stack & system design

Building on the motivation and contributions outlined in the introduction, this section describes the technology stack and system design that realize our digital-twin architecture. We first summarize the platform components and end-to-end flow, then detail the messaging and networking fabric, the predictive models, and the LLM guidance loop that together close the detection-to-action cycle.

The presented digital-twin platform combines multiple complementary technologies to enable continuous monitoring and maintenance forecasting in industrial settings. The platform comprises the following core elements:

- UE5: A modern game engine delivering high-resolution rendering and modeling capabilities for immersive virtual environments.
- Unity: A real-time 3D engine with native AR support for visualization and interaction on head-mounted devices and mobile endpoints.
- MQTT Protocol: A lightweight publish/subscribe protocol for device-to-system communication, carried via InsightCM's built-in RabbitMQ (MQTT interface).
- Machine Learning Models: Random Forest classifiers for malfunction identification and maintenance prediction using historical and live sensor measurements.
- Large Language Models: LLMs that transform model predictions and context into concise, operator-ready maintenance instructions for AR and web clients.
- Database: Unified storage of time-series and metadata in a local MongoDB instance for analysis, retraining, and audit.
- Web Portal: A browser interface for asset status monitoring, historical retrieval, and collaboration.
- Programmable Power Supply: An Ametek DC supply enabling local and remote control of output voltage and current.

Together with the physical equipment, these elements form a comprehensive digital-twin platform (see Fig. 2).

The platform begins with a direct-drive motor instrumented with two accelerometers (vibration sensors). Measurements are published via MQTT to the broker and distributed to five destinations: a database service, a web interface, the UE5 environment, the Random Forest model, and the LLM.

The database system functions as the storage center for selected streams and metadata, enabling secure and authorization-controlled user access through the web interface. The web interface provides performance analytics and access to historical information and trend evaluation, while the digital-twin application renders immersive virtual environments. For improved maintenance prediction, sensor data is regularly compiled and forwarded to a Random Forest model, which processes the information to recognize and classify potential motor malfunctions. RF forecasts are then relayed through MQTT to an LLM, which analyzes the state and context (e.g., tachometer speed) to produce concise remediation instructions. These instructions are transmitted to both the digital-twin application and the web interface to maintain user awareness of motor status and required procedures, and

both UIs update animations and overlays to visualize fault presence and classification, improving operator comprehension.

A second testbed was created using an identical motor system with a different configuration of sensors, data acquisition tools, and software. In this study, the initial testbed was modeled in UE5 while the second was modeled using Unity's native AR capabilities. The second motor is instrumented with four accelerometers for vibration data, two thermocouples to measure ambient and motor temperature, and a tachometer to measure speed; two accelerometers are colocated with the first testbed on pedestal tops and two are mounted perpendicularly on the rear side. The second testbed also implements a different system architecture: the Random Forest and LLM run locally on the InsightCM server to reduce communication overhead and improve efficiency. The motor is hardwired to a programmable power supply with a built-in web server, which is reverse-proxied via a container on the InsightCM host; the same reverse proxy serves an HTTP stream from a USB camera for remote monitoring. Finally, a ZeroTier One virtual private network enables devices across disparate locations to behave as if on the same LAN through an encrypted overlay that handles NAT traversal and peer-to-peer connectivity.

Having outlined the components and the two physical instantiations, we next describe the messaging and networking fabric that stitches publishers and consumers together.

2.1. Digital twin system architecture

The digital twin platform employs a component-based architecture encompassing data collection, processing, analysis, and display modules. Using this modular framework allows for a greater degree of freedom in digital twin design, is consistent with the separation of responsibilities principle, and increases the number of microservice-based solutions that can be incorporated into a digital twin system.

The game engine layer is implemented via UE5 or Unity and provides the foundational environment in which virtual representations of physical equipment are instantiated and continuously updated using real-time sensor data. Sensor measurements are gathered from operational equipment and forwarded to consumers via MQTT, while a local service persists selected streams to MongoDB. At regular intervals, specific data samples are processed by pre-trained machine learning algorithms to forecast potential malfunctions or irregularities. When a fault is identified, notifications and overlays are presented to stakeholders through the headset application and web portal. The web portal also offers access to historical datasets, educational resources, and equipment performance indicators, enabling users to make well-informed decisions and enhance maintenance approaches. Moreover, fault identification results are forwarded to both the LLM and visualization layers.

After receiving the fault categorization, the digital twin application initiates a tailored animation for fault visualization. Concurrently, the LLM formulates procedural guidance based on the state predicted by the random forest. This enables maintenance personnel to observe the simulated environment to identify the fault nature, location, and required remediation procedures (see Fig. 3).

2.2. Data flow overview

Fig. 4 summarizes the end-to-end data and control paths in the system. Data are collected by acquisition tools (sensors and cRIOs) and transmitted to the monitoring software. Streams are published via MQTT and ingested by the digital-twin application and a pre-trained Random Forest model. The model predicts the motor state; the prediction is then consumed by the LLM via the monitoring system's built-in MQTT broker. The LLM generates instructions applicable to the predicted state and publishes them via MQTT to the digital-twin environment for real-time display. The inclusion of a programmable

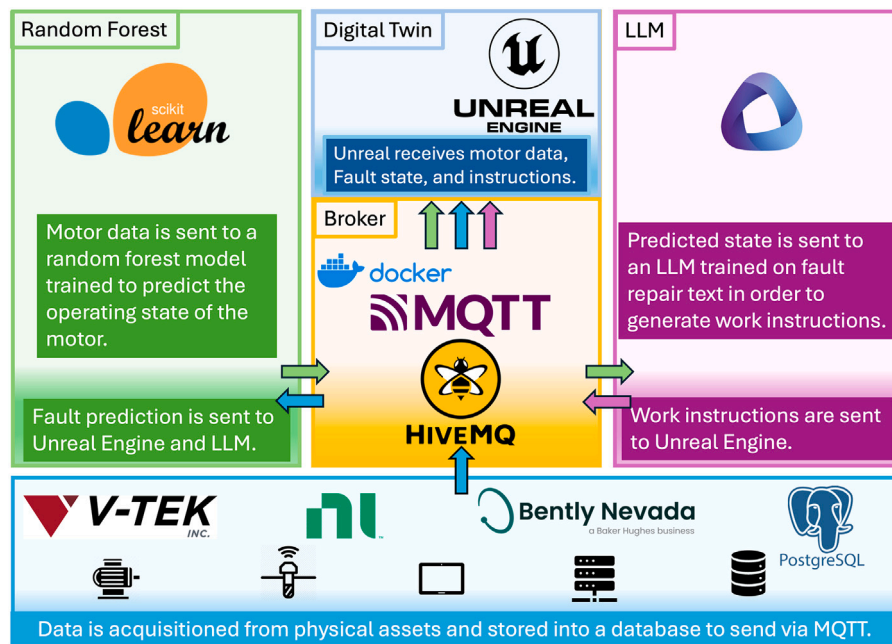


Fig. 2. High-level overview of system architecture and process flow.

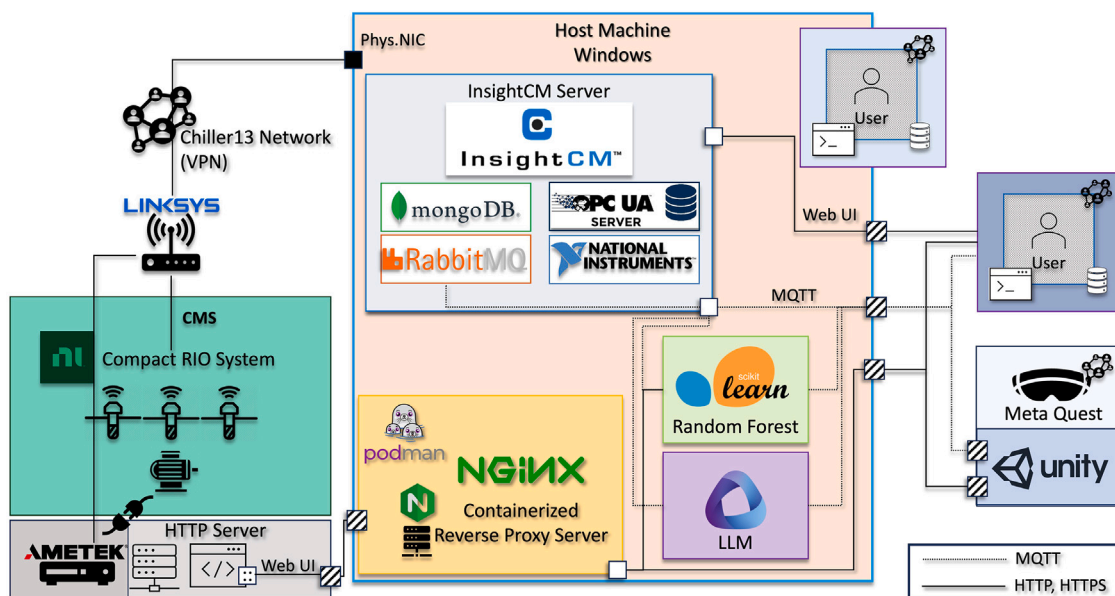


Fig. 3. Expanded overview of the modified architecture.

power supply enables two-way communication between the digital-twin environment and the physical world. Users can send commands from within the virtual environment to adjust the supply's output voltage and thereby change the motor speed. Continuous monitoring allows these changes to be observed in real time from both the application and the testbed.

2.3. Message queue telemetry transport

To interconnect heterogeneous producers and consumers across the twin, we rely on MQTT as the lightweight messaging layer that carries sensor readings, predictions, and guidance between services and user interfaces. MQTT is a standardized communication protocol developed for Internet of Things (IoT) applications by the Organization for the Advancement of Structured Information Standards (OASIS). It enables

efficient, low-latency communication among devices, including those with constrained memory, storage, or compute, and it operates well in limited-bandwidth environments. MQTT follows a publish/subscribe model with a broker coordinating communication among clients: publishers send messages to topics, and subscribers receive messages from topics of interest (MQ.T.T. Organization, 2024).

Like a client-server architecture, the main elements are clients and a broker. Clients use the publish-subscribe pattern, with the broker acting as the backend service that routes and filters messages. MQTT is compact and effective, offering scalability through optimized bandwidth use. Topic hierarchies are the broker's primary filtering mechanism. They define channels through which messages are published and subscribed to, reducing collisions and limiting the volume delivered to each subscriber.

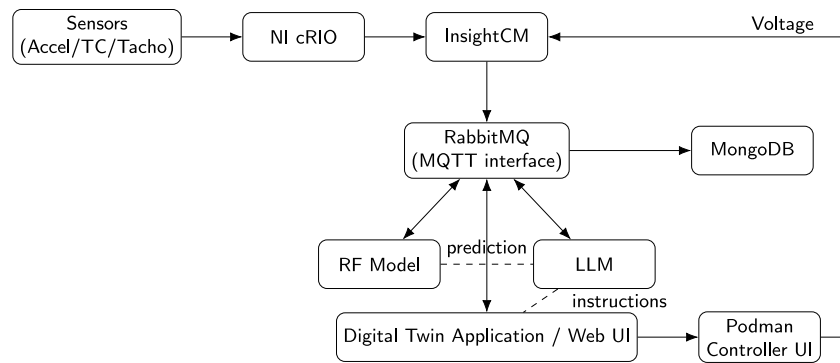


Fig. 4. Data and control flow in the digital twin system (InsightCM, RabbitMQ, RF, LLM, Podman, Digital Twin Application).

This structure organizes messages and supports quality-of-service guarantees. Brokers can be configured with authentication, encryption, and Transport Layer Security (TLS) to further secure IoT communications. Clients are assigned unique identifiers to preserve consistency and reliability.

2.3.1. Brokers

Modern versions of InsightCM provide MQTT capabilities for monitored assets by default and bundle RabbitMQ as the broker. Initially, we used a containerized HiveMQ broker external to InsightCM to provide flexibility and bridging to/from the InsightCM broker. To improve efficiency and reliability, the second testbed relies solely on InsightCM's built-in MQTT broker.

2.3.2. Client roles and topics

Head-mounted AR devices, the Random Forest (RF) classifier, the LLM service, and a local data ingestion service act as MQTT clients. A representative topic layout is as follows:

- InsightCM → `<device>/<asset>/<signal>` (e.g., vibration, temperature, tachometer) published by InsightCM integrated services
- Random Forest → `prediction/<[state]>` published by the RF model with current motor state predictions
- LLM → `<{instructions}>` published by the LLM with remediation steps corresponding to the predicted state
- `ui/events` optional channel for digital twin/browser UI acknowledgments or user selections

2.4. Networking and access

The software-defined overlay provided by ZeroTier allows endpoints—on-premises, edge, or cloud—to appear as if they are on the same LAN (ZeroTier, 2024). End-to-end encryption and peer-to-peer connectivity enable data and control traffic to traverse NATs and firewalls without a traditional VPN. In our architecture, ZeroTier provides a secure, low-latency fabric interconnecting sensor gateways, analytics engines, applications, and clients. This reduces deployment complexity and supports real-time coordination across distributed nodes.

2.4.1. Security

Devices join a private ZeroTier network (ZeroTier, 2024), allowing headsets and services to securely reach the RabbitMQ endpoint on the InsightCM server. Nodes require administrator authorization before joining. Authentication is enforced at the broker, and topic-level permissions constrain publish/subscribe rights per client. This design allows ZeroTier nodes to communicate securely across the Internet without requiring a shared physical LAN.

2.5. Data persistence

A lightweight consumer subscribes to selected topics and persists time-series and metadata to a local MongoDB instance. This enables offline analysis, model retraining, and auditability of predictions and instructions.

With the messaging, networking, and persistence layers in place, we next describe the predictive models that consume these streams and generate the fault states used throughout the system.

2.6. Predictive machine learning models

This study employs the Random Forest algorithm for predictive maintenance. Random Forests are ensemble methods that aggregate multiple decision-tree predictions to reduce variance and improve accuracy. They are widely used across domains such as image classification, speech recognition, and natural language processing (Cutler et al., 2012).

Various machine learning models were tested during preliminary experiments, including random forest, long short-term memory (LSTM), adaptive boosting (AdaBoost), and light gradient-boosting machine (LightGBM) models, with random forest and long short-term memory (LSTM) models demonstrating the highest performance.

Random forests handle complex tabular datasets with many potential features and signals critical in motor-fault assessment by aggregating the predictions of several decision trees trained on different subsets of the dataset. This ensemble structure makes them resilient to noise, outliers, and occasional missing data typical of IoT sensor streams, while reducing the risk of overfitting. Additionally, random forests can capture nonlinear interactions and provide measures of feature importance, making them effective and interpretable models for diagnostic and predictive maintenance applications (Sagi & Rokach, 2018).

Long short-term memory (LSTM) models are a type of recurrent neural network designed to capture temporal dependencies in sequential data by maintaining an internal memory of past states (Kizito et al., 2021). This makes them well suited for time-series and sensor data, where patterns evolve over time. By selecting retraining or forgetting information through gating mechanisms, LSTMs can model long-range dependencies while remaining robust to noise and irregularities commonly found in real-world data (Hagan et al., 2014).

The random forest model was selected for malfunction identification in this study based on its accuracy, robustness, adaptability, and interpretability. Additionally, random forests are generally considered to be less resource-intensive than neural networks. Preliminary experiments to compare the models highlighted this difference as the training times required for the LSTM were approximately ten times greater than that of a random forest using the same dataset. A comparison of the throughput and latency of the random forest and LSTM model is shown in 6(b).

Table 1
Compact comparison of DAQ setups.

Description	Testbed 1	Testbed 2
# Accelerometers	2 (top pedestals)	4 (2 top + 2 rear/perpendicular)
Additional Sensors	None	Tachometer + 2 Thermocouples
Acquisition Chain	Bently2300 → System1	NI-cRIO → InsightCM
Features (Accel)	Bias, Direct, DirectRMS, VelocityPeak, VelocityRMS, DerivedPeak	CrestFactor, Peak–Peak, RMS, TruePeak, HighFreq, DerivedPeak

Wherein the random forest was selected for the classification task, early experiments showcased the LSTM model outperforming the random forest model in regression tasks, such as predicting the RUL, though only slightly. Evaluation metrics of those experiments are shown in 4.

We use an 80/20 train–test split to classify multiple operating states, historically drawing approximately 5000 points per category from the System1 testbed (roughly one-hour runs with one faulty component installed per run). The second testbed uses InsightCM with NI cRIO and a seven-sensor array: four accelerometers (two colocated with prior positions on pedestal tops and two perpendicular on the same pedestals), two thermocouples (ambient, motor case), and a tachometer for speed.

The algorithm organizes features into multiple tree-based structures to learn a decision boundary that serves the full dataset. Over time, the tabular dataset is refreshed and the model can be periodically retrained with operational results, supporting continuous improvement without service interruption.

2.6.1. Transfer learning

Transfer learning is a technique in which a model initially trained on one task is adapted to perform a different but related task. By leveraging previously acquired knowledge, it reduces the time and computing resources required—especially beneficial when the second task has limited training data. Consequently, this approach can improve both model performance and generalizability. This study explores transfer learning across testbeds. Using identical motor systems in both settings provides a broader basis for comparison. As noted in the *System Design* section, the second testbed differs materially from the first. Table 1 outlines key differences in the data-acquisition process. We apply statistical analyses to assess similarity between the datasets captured from the two testbeds. We conduct experiments for zero-shot transferability and domain adaptation, and we test whether a model trained on either dataset generalizes better across environments. Specifically, this study explores the potential of utilizing transfer learning techniques to reduce the required overhead of creating or updating a machine learning model. Zero-shot transferability is tested first, in which a base model is created on a subset of data from a specified motor, and then tested against data from a disparate motor. If both motors produce similar data, a machine learning model could perform effectively without the need for re-training. Another method that is tested is domain adaptation, or mixed training. With this, a machine learning model is trained on small subsets of data from both motors, and then is used to predict on data from either motor.

2.6.2. Cross-testbed feature alignment

The definitions of extracted features differ between the System1 and InsightCM platforms. Although metrics such as root-mean-square (RMS) and spectral peaks can often be mapped between systems, other features—such as “Derived Peak”—lack standardized definitions and require rigorous validation; they may not be directly comparable. To mitigate this discrepancy, the analysis focuses on a common subset of features shared or derivable across both platforms. Preprocessing operations—including scaling, windowing, and filtering—are standardized to minimize domain shift and ensure a consistent feature space.

In summary, the Random Forest provides a robust, data-efficient baseline for fault classification across heterogeneous acquisition chains, while transfer learning and feature alignment techniques mitigate domain shift between testbeds. Building on these predictions, the next subsection details how an LLM transforms model outputs and context into concise, role-appropriate instructions.

2.7. Large language models

Our large language model (LLM) component targets practical manufacturing use cases in which operators and planners require timely procedural guidance while maintaining the required standards and procedures. In many facilities, a planning workgroup coordinates cross-disciplinary tasks (e.g., mechanics, electricians) and issues instructions and safety guidance. The LLM augments this process by transforming model outputs and context into concise, role-appropriate instructions.

The instructions and operations must follow strict guidelines based on the facilities roles and specific activities. In addition, the computing resources available in the field may be minimal at best compared to the needs of larger more complex LLMs. Thus, the instruction-generation task is therefore formulated as a constrained, fault-conditioned text completion problem, rather than a free-form natural language generation task. In this setting, semantic similarity or stylistic variation is insufficient; instructions must be reproduced exactly as validated to ensure compliance with established operating and safety procedures.

To meet these requirements, we employ the text-completion variant of the distilGPT-2 model, a lightweight autoregressive architecture with approximately 88.2 million parameters. This model was selected to enable fine-tuning and deployment on resource-constrained systems typical of industrial environments, while maintaining deterministic generation behavior. Larger instruction-tuned models, while state-of-the-art for open-ended language tasks, introduce unnecessary computational overhead and variability for this application. While many of these resources constraints can be off loaded to online cloud based resources, in facilities where security is a concern and net connections are restricted or not available, small LLMs such as the one applied here can be run locally on smaller systems.

During fine-tuning, each instruction set is converted into a single structured text string consisting of numbered procedural steps, preceded by the corresponding fault identifier. To address over generation, a common issue in smaller autoregressive models when using conservative token limits, a unique sentinel token is appended to the end of each instruction string. This allows the maximum generation length to be set sufficiently high to avoid truncation while enabling reliable identification of instruction termination during post-processing.

The model is fine-tuned for 24 epochs, a value determined empirically through iterative evaluation. The instructions used for fine-tuning consist of the specific fault states as well as instructions catered to those states. An example of this can be seen in Table 2. Performance is assessed by providing only the fault identifier as input and comparing the generated instruction sequence, parsed using the sentinel token, against the ground-truth instruction text. Evaluation is conducted using exact string match accuracy, which directly reflects the operational requirement that instructions be reproduced without omission, reordering, or modification. Using this evaluation framework,

Table 2
Sample of work instructions used for fine-tuning the LLM.

State	Instructions
“baseline”	“Operable condition, no action required.”
“angular misalignment”	“Loosen the coupling set screw on the motor shaft side of the coupling. Remove the motor from the motor pedestal on the left side of the MVAT. Remove the lift(s) from the motor pedestal. Re-install the motor on the motor pedestal. Be sure that the motor and rotor shafts are aligned.”

the model achieves 100% exact match accuracy on the instruction-generation task, indicating reliable and deterministic reproduction of validated instruction sets. This result demonstrates that a compact, fine-tuned language model can effectively support instruction generation in predictive maintenance workflows without reliance on large-scale generative models or cloud-based inference.

Multiple machine-learning models operate within the system and feed the digital-twin architecture. Beyond assisting shop-floor personnel, the twin also supports planners and managers who coordinate assignments. One deployed methodology uses historical work orders to train a model that predicts instruction templates for future tasks. To enable this, we first consolidated historically dispersed work-order data and built a live data-stream infrastructure that integrates multiple sources in real time. Using a form template (Quasar JavaScript framework), commonly used values are pre-filled while preserving editability for planners, yielding dynamic, consistent instructions rather than static printouts.

In parallel, a Random Forest algorithm identifies rotor faults from testbed data. Bearing faults, for instance, produce distinctive feature sets; subsequent recurrences can be classified accordingly. These analytical pipelines run concurrently and feed the digital-twin system. Operationally, the LLM runs on the InsightCM server alongside the Random Forest classifier. The classifier publishes predicted motor states to the built-in RabbitMQ (MQTT interface). The LLM subscribes, composes a prompt from the detected state and recent sensor metadata (e.g., tachometer speed), and generates concise remediation steps. It publishes the resulting instructions to an MQTT topic that AR headsets and browser-based UIs consume to display guidance and synchronize animations/overlays.

Together, the RF predictor and the LLM close the loop from detection to action within the twin. We next operationalize this design in two physical testbeds and report results from a case study deployment.

3. Implementation and case study

This section operationalizes the system design in two complementary testbeds and evaluates the end-to-end loop from sensing to guidance. We extend prior work by implementing predictive maintenance algorithms across distinct industrial monitoring scenarios and by integrating AR/web delivery for operator-ready instructions.

Testbed 1: Accelerometer-based vibration monitoring. The first testbed features a rotor system with variable-speed operation, simulating conditions akin to factory settings. Two accelerometers are mounted on bearing pedestals and connected to a Bently Nevada 2300 series monitor running System 1 machine-management software. The system captures six vibration metrics—bias, derived peak, direct, direct RMS, velocity peak, and velocity RMS—providing a comprehensive dataset for training and validation.

Testbed 2: Multi-sensor monitoring and control system. The second testbed introduces additional complexity with four accelerometers—two colocated as in the first testbed and two mounted perpendicularly on the rear of the bearing pedestals—plus a tachometer for speed and two thermocouples monitoring ambient and motor-case temperatures.

These sensors interface with an NI cRIO device connected to a computer running InsightCM software. The system captures six vibration metrics: crest factor, derived peak, peak-to-peak, RMS, true peak, and high-frequency components, enabling a more nuanced analysis of rotor health.

Data integration and model training. Before training the models, datasets underwent a series of preprocessing steps to ensure the quality of the data and suitability for the predictive models. The datasets were carefully examined to identify missing or null values, outliers, imbalances, and inconsistencies.

Early-stage experiments revealed that the random forest models performed better on data that had not been normalized. Due to the random forest’s robustness to unstandardized data, it was determined that the overhead during preprocessing could be reduced by neglecting to normalize datasets before exposing it to the models.

Data from both testbeds were integrated into a unified digital-twin architecture for real-time monitoring and predictive analytics. Integration involved standardizing preprocessing steps—scaling, windowing, filtering—to mitigate domain shift between systems. Models were trained on historical data from both testbeds to develop predictive strategies applicable across diverse environments.

Continuous model retraining. Maintaining historical motor-state records enables periodic Random Forest retraining to accommodate component wear and improve performance. To avoid service interruption, a duplicate of the incumbent model is archived; the new model is trained on selected labeled data and evaluated against the archived copy. If the new model meets or exceeds performance, it is promoted; otherwise, it is discarded.

Procedural guidance via LLM. The LLM processes the predicted fault type to produce an integer:state encoding and procedural guidance to address the fault. The state encoding and instructions are transmitted through RabbitMQ’s MQTT interface and received by both the Unity AR application and the browser interface. Both platforms use the state to activate the appropriate virtual model and animation, delivering a visual depiction of the current fault alongside instructions on the web UI and within an AR overlay widget.

System architecture and remote control. The implementation centers on deploying an end-to-end algorithm for maintenance prediction. The current system employs the InsightCM-based testbed with NI cRIO and a nine-sensor array (four accelerometers, four thermocouples, one tachometer). One thermocouple measures ambient temperature, one is affixed to the motor case, and two monitor an auxiliary hot/cold loop (not fault-relevant). The tachometer captures speed, which immediately reflects user-issued voltage changes. Head-mounted AR devices and workstation services join a ZeroTier virtual network to securely reach the InsightCM server’s built-in RabbitMQ broker. An MQTT client on each headset subscribes to predictions and instructions. The industrial controller’s web interface is served from a Podman container on the InsightCM server, allowing the AR headset or a browser to remotely adjust motor voltage; resulting speed changes are observable in real time via the tachometer and AR overlays.

Safety and failure handling. To support safe operation of the system, the architecture is designed around a human-in-the-loop validation process. When a non-baseline state is predicted, a reliability engineer is notified and must explicitly confirm or reject the predicted fault classification before any maintenance workflow is initiated. If a prediction is rejected, the engineer may supply the correct motor state, and the associated sensor data are flagged and retained for inclusion in subsequent model retraining cycles. This feedback prevents the automated propagation of incorrect diagnoses, and it facilitates improvement of the predictive model by emphasizing difficult or misclassified cases during retraining. These safeguards ensure that model outputs remain advisory rather than authoritative, preserving human oversight while progressively improving model robustness.

Table 3
Example data notations with samples and descriptions.

System1	Sample	InsightCM	Sample
Feature	Sample	Feature	Sample
timestamp Time of sensor data capture	03/21/2024 15:46	datetime Time of sensor data capture	2025-08-01 13:18
Channel 1 Bias Inherent bias prior to calculation	-12.032	Accelerometer 1 Crest Factor True Peak/RMS	1.633
Channel 1 Derived Peak Magnitude of rotor displacement	0.773	Accelerometer 1 Derived Peak $RMS \times \sqrt{2}$	0.255
Channel 1 Direct RMS Root mean square of signal	0.547	Accelerometer 1 RMS Root mean square of signal	0.18
Channel 1 Velocity RMS Velocity of rotor displacement	0.092	Accelerometer 1 True Peak $\max(P_{\max} , P_{\min})$	0.295
Channel 1 Velocity Peak Peak of Velocity	0.233	Accelerometer 1 Peak-Peak $P_{\max} - P_{\min}$	0.562
Status Operating state of motor	imbalance	Status Operating state of motor	baseline

3.1. Fault categories

Data are collected under controlled operating conditions for the following states in the current testbed: *baseline*, *imbalance*, *looseness*, and *angular misalignment*. A looseness condition can be established by loosening either of the two bearing pedestals positioned on the rotor base plate. This study defines standardized processes for inducing these faults in an effort to ensure conformity across state-changing experiments. The random forest algorithm processes data for each classified state to forecast the current condition using labeled failure data.

Data collection types are displayed in Table 3. The “Channel 1” and “Accelerometer” designations indicate the first sensor of that type registered in the system and help map data to physical positions on the motor. Measurements are duplicated via a redundant sensor on the opposite side for comparison. While the example refers to the first channel, the second channel captures a corresponding data point for each record. This redundancy enables precision checks and comparative analysis across sensors.

Most measurements in Table 3 use millimeter units; parameters such as velocity incorporate time in seconds. Bias remains largely constant across runs because it is determined during the initial motion. During a typical run, axial displacement increases as the rotor accelerates to operating speed, then stabilizes once the rotor reaches steady-state, reducing measured variability.

Exploratory data analysis was performed on both datasets, and statistical methods were employed to compare them. Distributions and kernel density estimates (KDEs) for the two testbeds are shown in Fig. 5, providing an initial qualitative comparison of their statistical characteristics. The shifted distributions and the differing variance profiles of the same feature from nominally identical motors with equivalent sensor placement reveal a pronounced domain mismatch, motivating the need for mixed-domain training and feature-alignment techniques. To quantitatively assess whether the two datasets were statistically similar, a two-sample z-test was performed using a significance level of $\alpha = 0.05$, yielding $z = 4.6$ and $p = 4.2 \times 10^{-6}$. These results indicate that the two datasets differ significantly, providing strong evidence against the null hypothesis of similarity. Fig. 6(a) further highlights differences in the magnitudes between the two testbeds’ features when comparing their means. Further investigation is required to determine whether these differences arise from physical environmental factors or from sensor and data acquisition tooling.

The bar chart shown in Fig. 7 demonstrates the test-accuracy results for the random forest model under two distinct training strategy configurations. The “Mixed Training” strategy yielded a near perfect test accuracy, whereas the zero-shot strategy produced only approximately 44.03% accuracy. This stark contrast underscores the importance of combining data from both domains rather than relying on a single

Table 4
Regression model performance comparison.

Regression model	R^2	MSE	MAE
Random Forest	0.999	352902310.06	1196.61
LSTM	0.999	136469601.07	1885.17
Light GBM	0.998	466562635.20	6546.33
AdaBoost	0.965	13816713711.85	93513.19

domain’s dataset in isolation. By visualizing the raw numeric results in a side-by-side format, the figure makes the performance disparity immediately accessible to the reader.

The dot plot on the right of Fig. 7 further contextualizes the relationship between training-sample size and model performance for the same two strategies. Here, the Mixed Training configuration aligns directly with the 100% accuracy threshold and is trained on a moderate number of samples (6179), whereas the Chiller-only strategy, despite utilizing a much larger sample set (17,939), yields substantially inferior accuracy (44%). These results suggest that domain-alignment—training across coherent operational domains—can outweigh mere sample volume in determining model efficacy. Accordingly, the empirical evidence supports the conclusion that ensuring sample diversity and domain relevance may be more beneficial than simply increasing data quantity (see Fig. 9).

Rankings of the feature importance for the classification models are shown in 8. These findings provide further insight into the underlying logic used by the models based on their respective training dataset, and potential challenges in generalizing each model to perform on the disparate dataset. An example of this is the reliance of the *Chiller* model on Thermocouple 2, wherein the dataset used to train the *S1* model did not contain any thermocouple data.

The two confusion-matrix panels illustrate the dramatic contrast in classification performance between the random forest model trained on mixed-domain data and a zero-shot transfer scenario. In the mixed training case, where the model is trained on combined data from both the “Chiller” domain and the original System 1 domain (6179 samples), the classifier achieved perfect separation—5639 : 5639 baseline cases and 4367 imbalance cases were correctly predicted, no misclassifications in the validation set. Conversely, the zero-shot transfer scenario—trained only on the Chiller domain and tested on entirely unseen System 1 data—yielded an accuracy of just 0.12%. Nearly all 5639 baseline cases were misclassified as imbalance, and 10 imbalance cases were misclassified as baseline. This stark divergence underscores the model’s inability to generalize effectively across domains without target-domain training data or other adaptation methods.

Moreover, the results highlight the critical importance of domain alignment in predictive maintenance applications. Despite the larger

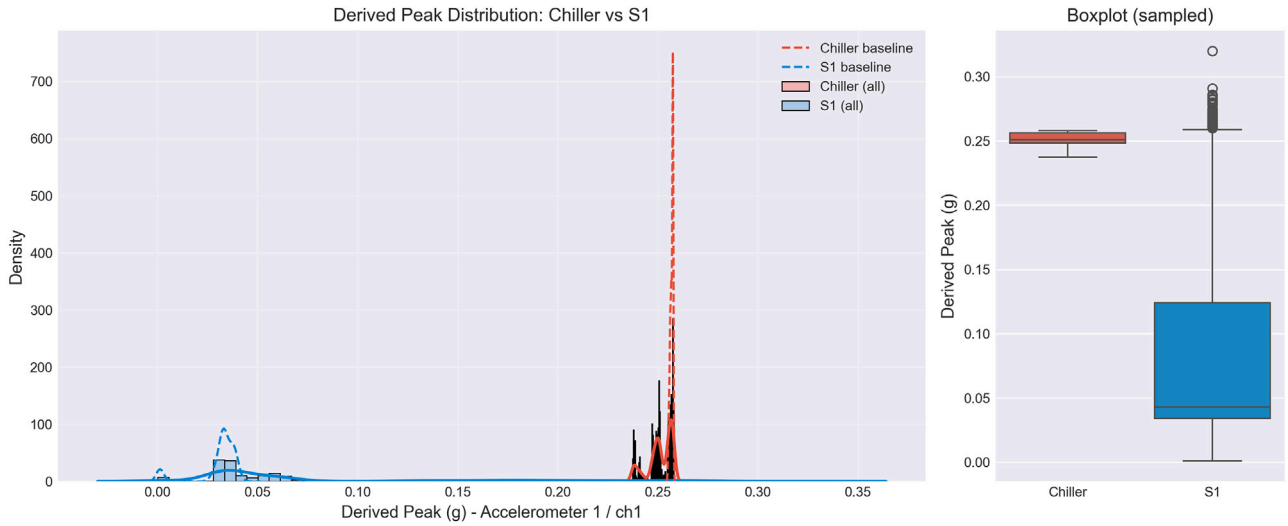
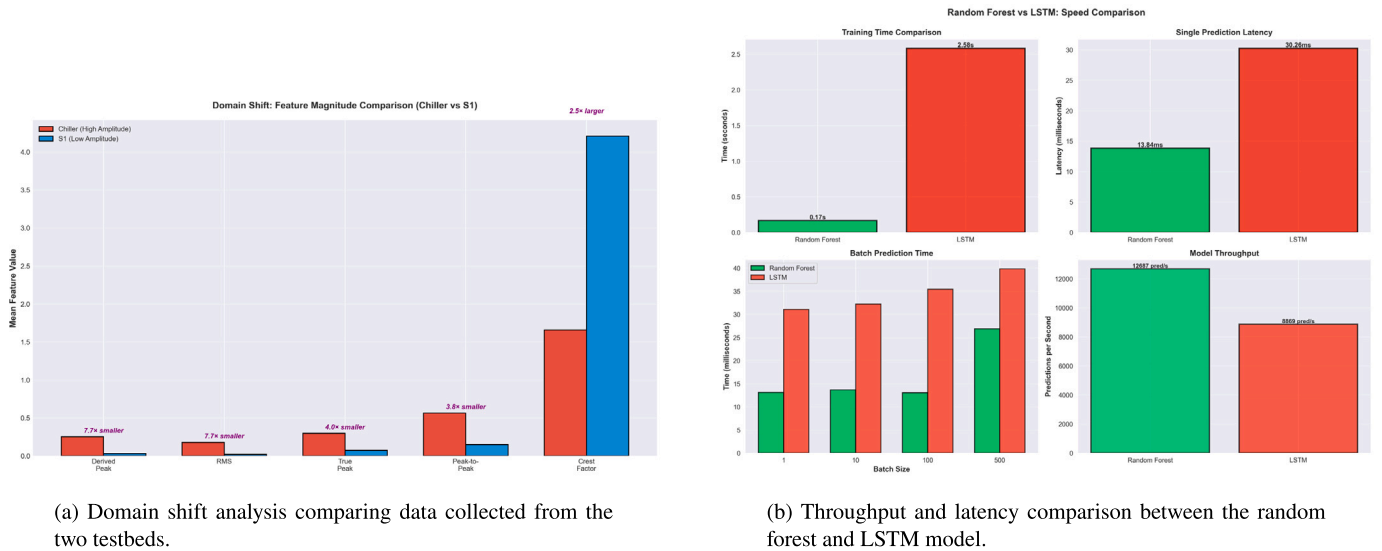


Fig. 5. Distributions of the derived-peak feature from the same sensor location on both testbeds.



(a) Domain shift analysis comparing data collected from the two testbeds.

(b) Throughput and latency comparison between the random forest and LSTM model.

Fig. 6. Comparative analysis.

sample size available in the zero-shot scenario (over 5600 Chiller-only training samples compared to 6000 mixed samples), the model's performance collapsed when confronted with unseen target-domain characteristics. The mixed-domain model, by contrast, leveraged cross-domain diversity to achieve flawless classification on this dataset. These results demonstrate that increasing the volume of training data from a single domain does not compensate for the absence of representative samples from the target domain. Effective transfer thus demands either the inclusion of target-domain data or the application of formal adaptation techniques, which may have substantial implications for industrial-scale deployment of machine-learning systems in heterogeneous operational environments (see Table 5).

3.2. Machine learning implementation

From an operational perspective, planners are responsible for equipping craft personnel with the necessary tools, safety gear, and procedural guidance to complete their designated tasks. These instructions are compiled by individual planners tailored for distribution to their

Table 5

Evaluation metrics of random forest classification models.

Base model	Accuracy	Precision	Recall	F1	Cohen's Kappa (k)
Chiller	0.98	0.99	0.99	0.99	0.98
S1	0.99	0.99	0.99	0.99	0.99
Transfer Method					
Zeroshot	0.43	0.21	0.5	0.29	0
Mixed	0.99	0.99	0.99	0.99	0.99

respective teams. To enable the LLM to generate such guidance, historical work-order records were analyzed. These data volumes acquired from both testbeds provided a broad foundation from which the LLM learned prevailing patterns of planner-authored instructional text.

The LLM supports this challenge by aggregating the planners' historical work order language and producing draft instructions that mirror their established writing style. In doing so, it accelerates the creation of work-order documentation, transforming the planner's role from initial authorship to focused refinement of largely completed texts. In addition

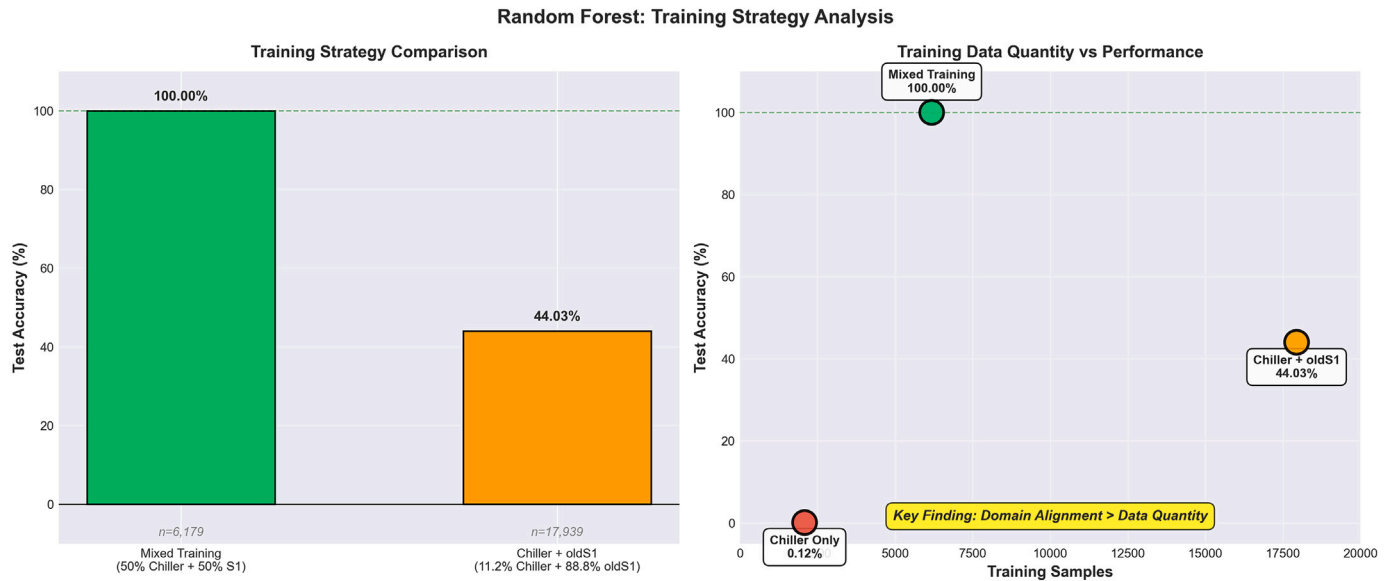


Fig. 7. Random Forest evaluation results: mixed-domain training vs. single-domain (Chiller-only) strategy.

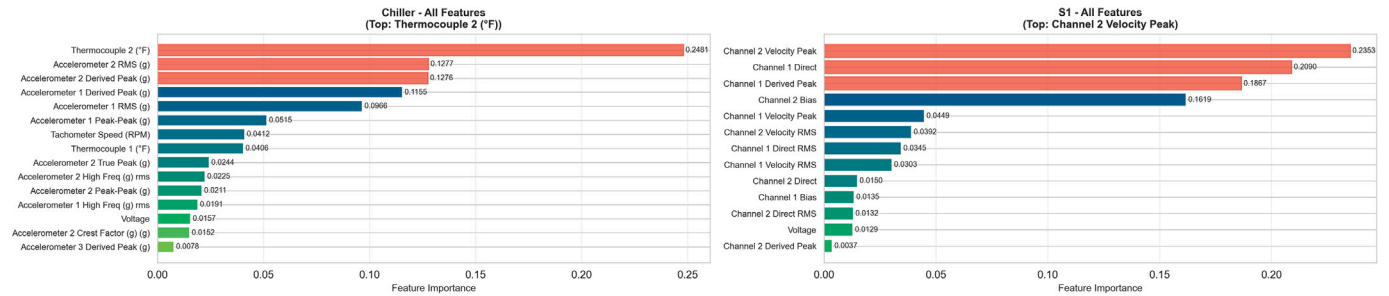


Fig. 8. Comparison of the feature importance rankings between the models.

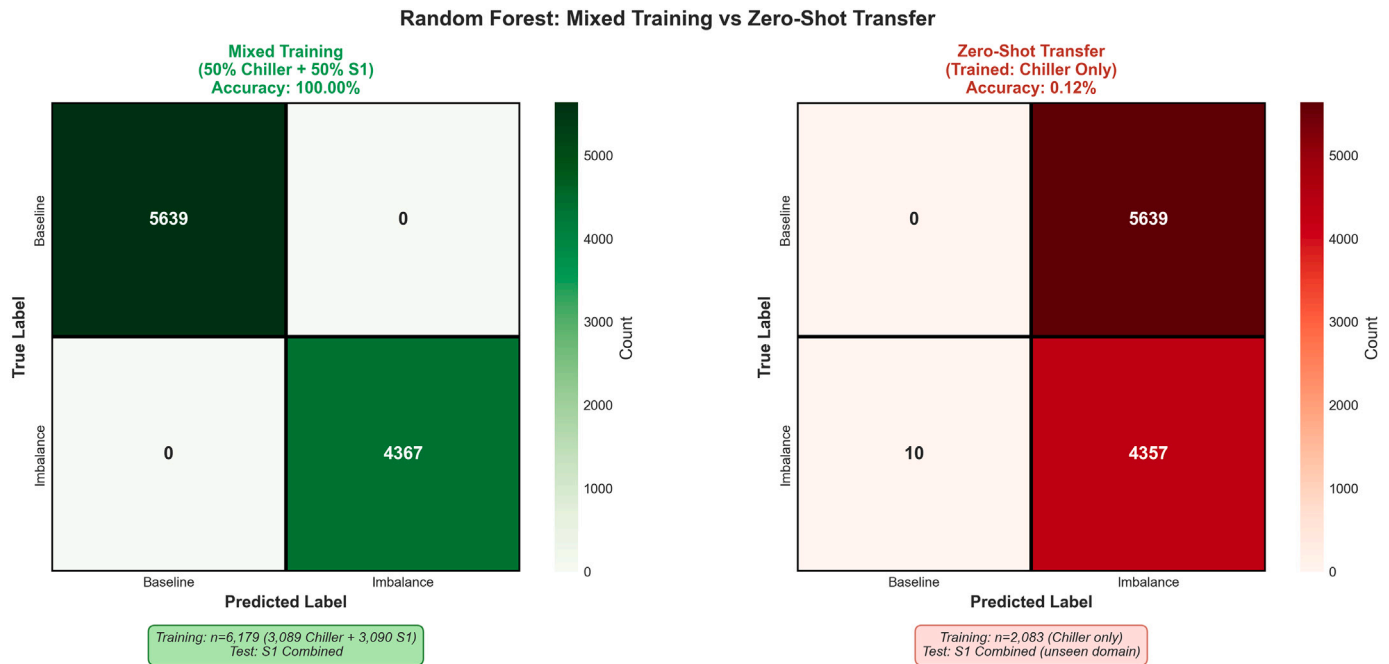


Fig. 9. Left: Confusion matrix of the random forest model using a mixed training methodology. Right: Confusion matrix of the random forest model using a zero-shot methodology.

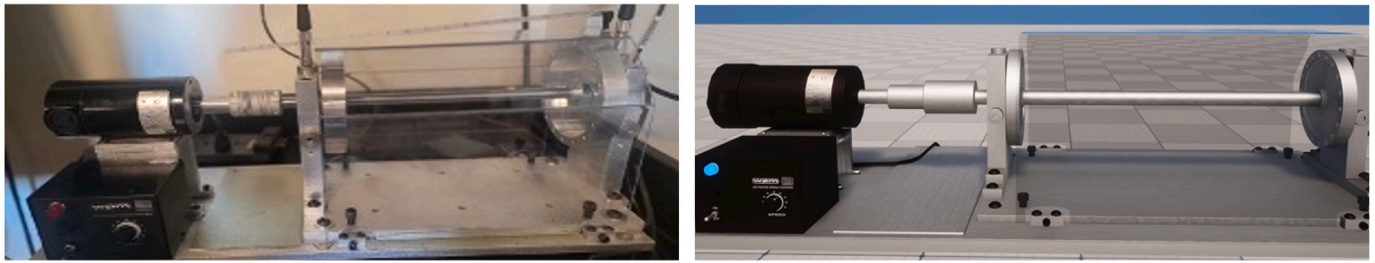


Fig. 10. S1 test bed motor (left) virtually modeled in the UE5 environment (right).

to generating text, the system integrates with an augmented-reality and web interface; the state codes produced by the LLM are transmitted via the MQTT interface to both the Unity AR application and the browser UI, enabling dynamic virtual models and overlays to reflect current conditions.

Procedurally, planners must adopt a framework that addresses their direct responsibilities as well as the functions of ancillary governance teams that approve major corporate initiatives. Communication across physical and digital channels remains essential; however, planners must access multiple documents: safety procedures, application-specific protocols, and regulatory references that are distributed among various digital and physical repositories. The deployment of an AI-powered assistant offers a solution: using an LLM fine-tuned on multi-model industrial documents and work histories, the assistant can suggest relevant procedural texts in response to natural-language queries, thereby accelerating document retrieval and providing context-aware work-order guidance.

In the implemented system, sensor data from both testbeds are collected and processed through distinct acquisition chains. In the first testbed, vibration data from the dual-accelerometer setup are transmitted via System 1 and exposed through the local RabbitMQ broker. In the second testbed, sensor data from the multi-sensor (four accelerometers, two thermocouples, one tachometer) InsightCM configuration is likewise streamed into the broker. Selected streams from both systems are persisted to a MongoDB repository by a local client. At regular intervals, consolidated data are forwarded to the Random Forest model, which produces a predicted state. That output is then published via MQTT and ingested by the LLM.

The LLM accepts the fault prediction and contextual metadata as input and generates both an integer-based state encoding and procedural instructions to address the diagnosed fault. These outputs are transmitted over MQTT and consumed by the digital twin application and the web-based interface. Both platforms use the state encoding to trigger the corresponding virtual model and animation, which visually depict the fault condition. The instructions generated by the LLM appear on the web-portal display and as an overlay widget within the AR headset (see Fig. 10).

When the decoded state deviates from baseline, the web interface raises an alert and dispatches email notifications to condition-based reliability engineers, prompting verification of the model's prediction. The notification includes a link to the web portal and presents the underlying data points along with options to confirm or reject the predicted state. Upon user confirmation, the prediction is stored in the database and the appropriate maintenance workflow is initiated. If users reject the state, they may specify an alternate classification — either baseline operations or a distinct fault state, thereby appending the revised label to the database for future model training.

Maintaining a historical chronology of motor-state records from both testbeds enables continuous re-training of the random forest model to accommodate machine component degradation over time and to enhance overall predictive performance. To ensure reliability, a duplicate of the incumbent model is archived prior to re-training. The model is retrained using selected labeled data from the repository and

then evaluated against the archived copy. If the retrained model's performance metrics meet or exceed those of the incumbent, it is deployed in place of the previous model; if performance degrades, the newly trained model is discarded (see Figs. 11 and 12).

A case study evaluation of the digital twin system was conducted within an industrial setting exploring a motor-driven asset. Sensors installed on both testbed configurations collected vibration data, which was processed and visualized in real-time via the integrated framework. The machine-learning model accurately detected emerging faults, enabling condition-based actions, reducing operational downtime, and elevating operational efficiency. Furthermore, the web portal furnished operators and maintenance personnel with actionable insights into asset performance and maintenance requirements, thereby facilitating cost reduction and process improvement.

Taken together, these results illustrate a practical path to delivering innovative, operator-centric maintenance support: predictions derived from mixed-domain training drive synchronized AR/web overlays, while the LLM translates model outputs into concise actions. We conclude with a summary of key takeaways and directions for scaling and generalization.

4. Limitations, discussion, and conclusions

4.1. Limitations and practical considerations

While the proposed digital-twin architecture demonstrates the feasibility of tightly integrating game engines, MQTT-based communication, supervised learning, and LLM-driven guidance for predictive maintenance, several limitations should be acknowledged. First, the current evaluation was conducted on a limited number of laboratory-scale testbeds. Although the modular design supports extensibility in principle, its scalability has not yet been validated at fleet or enterprise scale. Larger deployments would introduce additional challenges related to message throughput, synchronization of distributed services, data persistence, cybersecurity, and end-to-end latency management.

Second, sustained real-time operation across a large number of assets may require substantially greater computational resources. While the Random Forest classifier and lightweight LLM were selected to balance predictive performance with efficiency, coupling continuous inference with high-fidelity visualization and potential AR interfaces could increase reliance on edge or cloud infrastructure, thereby raising system complexity and operational cost.

Third, the current implementation depends on specific software platforms and acquisition tooling, most notably InsightCM and its integrated MQTT infrastructure. Although these tools provide reliable industrial monitoring capabilities, they may constrain reproducibility and limit adoption in environments built on alternative condition-monitoring systems. Reducing reliance on proprietary components remains an important step toward broader accessibility and long-term sustainability.

Finally, the cross-testbed experiments reveal that generalization cannot be assumed even for nominally similar assets. Differences in

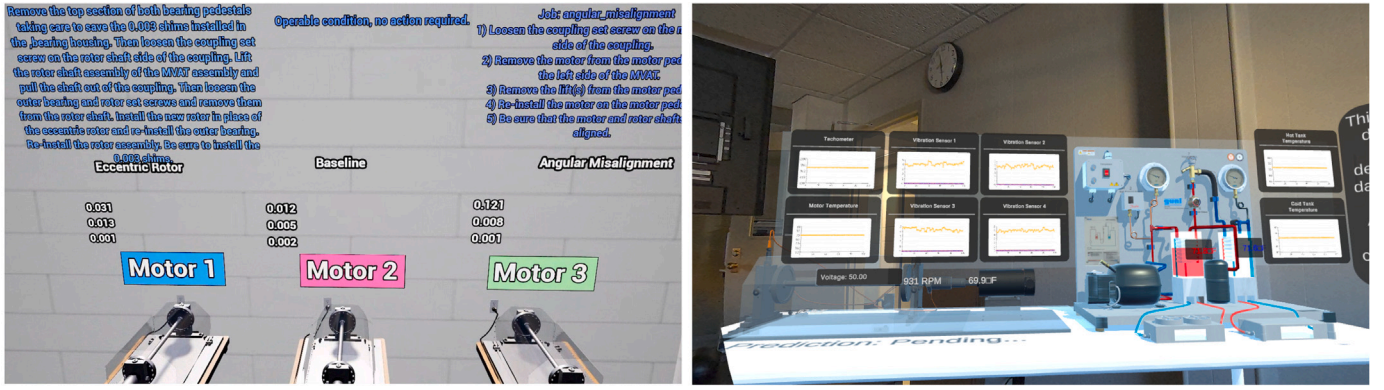


Fig. 11. Left: Demo of the UE5 VR environment. Right: Demo of the Unity AR overlay.

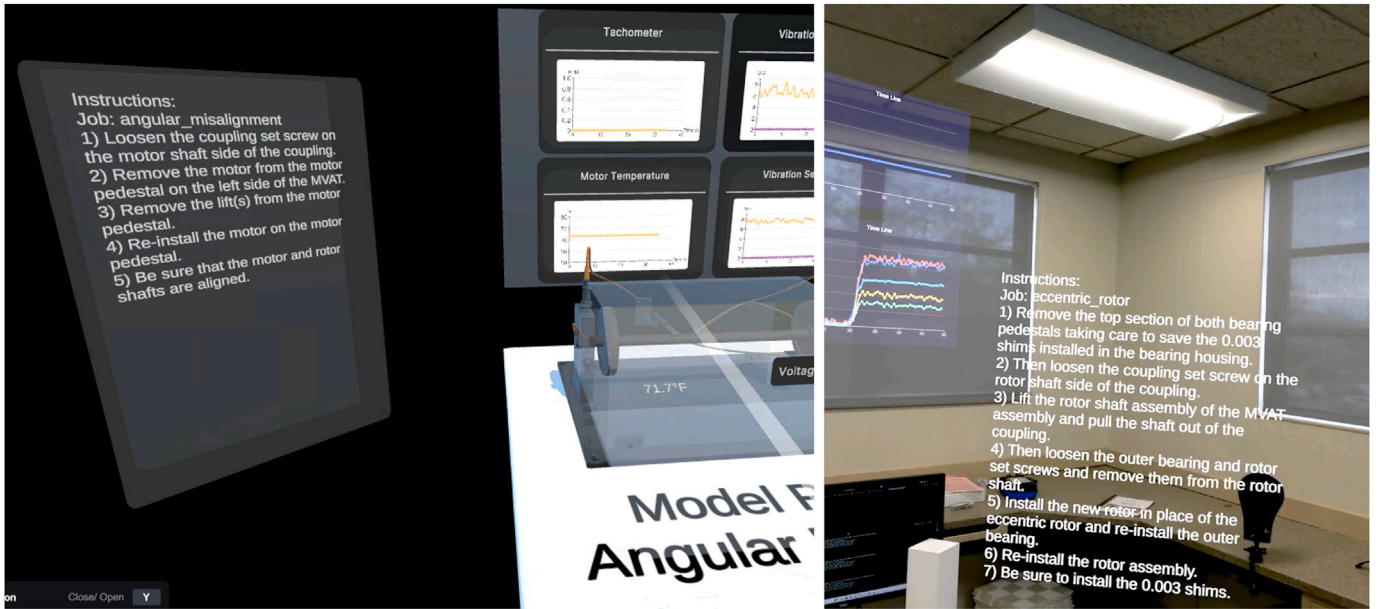


Fig. 12. Left: Example of the instructional text overlay in the Unity development environment. Right: Example of the instructional text overlay presented in AR.

sensor configurations, feature definitions, and operational regimes introduce measurable domain shift, leading to degraded zero-shot performance. These findings highlight the necessity of representative target-domain data or formal domain-adaptation strategies when transferring models across sites. From a practical standpoint, real-world deployment further entails organizational and economic considerations, including hardware investment, software licensing, cybersecurity compliance, and workforce training. Establishing trust in automated predictions and LLM-generated guidance will require transparent validation workflows and human-in-the-loop oversight.

4.2. Discussion: Digital twin & LLMs

Despite the limitations, this work presents a practical and modular digital-twin platform that unifies real-time messaging, supervised learning, immersive visualization, and on-edge LLM reasoning to support condition monitoring and maintenance guidance. The Random Forest classifier demonstrated robust performance when trained on mixed-domain data, whereas zero-shot transfer across heterogeneous testbeds proved unreliable, reinforcing the importance of domain alignment in industrial machine-learning applications. The LLM component successfully translated model predictions and contextual system states into concise, operator-ready instructions that can be delivered through web

or AR interfaces, illustrating the value of combining predictive analytics with natural-language decision support.

At the same time, the interpretation of guidance varies across operator roles, experience levels, and operational contexts. Improving accessibility, tailoring instructions to different crafts, and supporting varying degrees of technical depth remain important challenges. Systematic collection of structured feedback from end users will be critical for refining interface design, improving instructional clarity, and enhancing trust in automated recommendations.

4.3. Conclusions and future directions

In summary, this study demonstrates that a digital-twin platform for continuous monitoring and maintenance prediction can enhance industrial asset management by enabling proactive interventions, reducing unplanned downtime, and improving operational performance. By integrating simulation, real-time data streaming, machine learning, and LLM-based guidance within a unified architecture, the proposed framework bridges the gap between fault detection and actionable decision support.

Future work will focus on several key directions: (i) the development of formal domain-adaptation methods to support robust cross-site generalization and plug-and-play deployment in new environments;

(ii) continuous-training workflows with drift detection and human-in-the-loop validation to enable CI/CD-style model maintenance without compromising reliability; (iii) expanded AR interactions and safety-aware overlays, such as visual highlights or warning reminders, to promote situational awareness and adherence to safety procedures; and (iv) scaling the architecture to larger fleets and additional sensor modalities, enabling simultaneous modeling of multiple machine types and fault states.

Collectively, these directions chart a practical path toward deploying operator-centric digital twins at industrial scale—grounded in modular and extensible components, validated across domains, and designed to transform predictive insight into timely, trustworthy action.

CRedit authorship contribution statement

John Williams: Writing – review & editing, Writing – original draft, Software, Methodology. **Gerald Jones:** Writing – review & editing, Software, Methodology. **Tom Berg:** Supervision, Project administration, Funding acquisition, Conceptualization. **Luke Birt:** Supervision, Resources, Project administration. **Ashley Stowe:** Resources, Project administration, Conceptualization. **Xueping Li:** Writing – review & editing, Supervision, Methodology, Funding acquisition, Conceptualization.

References

- Clausen, C. S. B., Ma, Z. G., & Jørgensen, B. N. (2022-12-21). Can we benefit from game engines to develop digital twins for planning the deployment of photovoltaics? *Energy Informatics*, 5, 42.
- Cutler, A., Cutler, D. R., & Stevens, J. R. (2012). *Random forests* (pp. 157–175). New York, NY: Springer New York.
- Duan, J.-G., Ma, T.-Y., Zhang, Q.-L., Liu, Z., & Qin, J.-Y. (2023). Design and application of digital twin system for the blade-rotor test rig. *Journal of Intelligent Manufacturing*, 34(2), 753–769.
- Epic Games (2024). Unreal engine. URL <https://www.unrealengine.com>. (Accessed 25 April 2019).
- González-Herbón, R., González-Mateos, G., Rodríguez-Ossorio, J. R., Domínguez, M., Alonso, S., & Fuertes, J. J. (2024). An approach to develop digital twins in industry. *Sensors*, 24(3), 998. <http://dx.doi.org/10.3390/s24030998>, URL <https://www.mdpi.com/1424-8220/24/3/998>.
- Hagan, M. T., Demuth, H. B., Beale, M. H., & De Jesús, O. (2014). *Neural network design* (2nd ed.). Martin Hagan.
- Kamal, Z., Taghaddos, H., & Karimi, H. (2021). BIM-based maintenance management system for healthcare facilities. *Journal of Performance of Constructed Facilities*, 35(4), Article 04021036. [http://dx.doi.org/10.1061/\(ASCE\)CF.1943-5509.0001604](http://dx.doi.org/10.1061/(ASCE)CF.1943-5509.0001604).
- Kang, Z., Catal, C., & Tekinerdogan, B. (2020-11). Machine learning applications in production lines: A systematic literature review. *Computers & Industrial Engineering*, 149, Article 106773. <http://dx.doi.org/10.1016/j.cie.2020.106773>.
- Karabacak, M., & Margetis, K. (2023). Embracing large language models for medical applications: opportunities and challenges. *Cureus*, 15(5).
- Kizito, R., Scruggs, P., Li, X., Devinney, M., Jansen, J., & Kress, R. (2021). Long short-term memory networks for facility infrastructure failure and remaining useful life prediction. *IEEE Access*, 9, 67585–67594.
- Li, X., Swanson, C., Jones, G., Berg, T., Birt, L., Seals, M., & Williams, J. (2023). A machine learning approach to predictive maintenance: Remaining useful life and motor fault analysis. *CIE50 Conference*.
- Liu, J., Xia, C. S., Wang, Y., & Zhang, L. (2023). Is your code generated by ChatGPT really correct. Rigorous Evaluation of Large Language Models for Code Generation. CoRR, Abs/2305.01210.
- Mesko, B., & Topol, E. J. (2023). The imperative for regulatory oversight of large language models (or generative AI) in healthcare. *NPJ Digital Medicine*, 6(1), 120.
- MQ. T. T. Organization (2024). The standard for IoT messaging. URL <https://mqtt.org/>. (Accessed 28 May 2024).
- Nath, A. G., Udmale, S. S., & Singh, S. K. (2021). Role of artificial intelligence in rotor fault diagnosis: A comprehensive review. *Artificial Intelligence Review*, 54, 2609–2668.
- National Academies of Sciences, Engineering, and Medicine (2023). *Opportunities and Challenges for digital twins in engineering: Proceedings of a workshop—in brief*. Washington, DC: The National Academies Press, <http://dx.doi.org/10.17226/26927>.
- Rajapakse, A. D., Gomez, F., Nanayakkara, K., Crossley, P. A., & Terzija, V. V. (2009). Rotor angle instability prediction using post-disturbance voltage trajectories. *IEEE Transactions on Power Systems*, 25(2), 947–956.
- Sagi, O., & Rokach, L. (2018). Ensemble learning: A survey. *WIREs Data Mining and Knowledge Discovery*, 8(4).
- Starly, B., Koprov, P., Bharadwaj, A., & Batchelder, T. (2023). “Unreal” factories: Next generation of digital twins of machines and factories in the metaverse. *Manufacturing Letters*.
- Sun, Y., Zhang, Q., Bao, J., Lu, Y., & Liu, S. (2024). Empowering digital twins with large language models for global temporal feature learning. *Journal of Manufacturing Systems*, 74, 83–99.
- Vyas, N. S., & Satishkumar, D. (2001). Artificial neural network design for fault identification in a rotor-bearing system. *Mechanism and Machine Theory*, 36(2), 157–175.
- ZeroTier, I. (2024). ZeroTier. Virtual network overlay for secure connectivity. URL <https://www.zerotier.com/>.